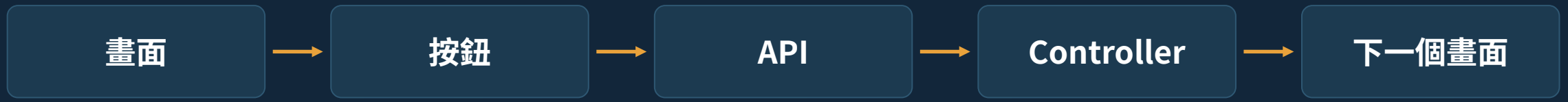
後端技術分享 • 2026.10.30

ScreenTrace

從原始碼重建畫面、流程與 API 的關係



01

認識 ScreenTrace

它要解決什麼問題、能做什麼、怎麼使用

目的

接手舊系統時，最常卡住的三件事

接

接手舊系統

文件早已過時，畫面和程式的對應只留在原開發者腦中

盤

功能盤點

需求方用畫面溝通，後端用 URL 與 Controller ，雙方缺少共同語言

改

決定改版範圍

要先知道哪些畫面、按鈕、 API 還在用，才能決定遷移什麼

目標：把「人工追程式」變成可重現、每一步都能回到原始碼的分析結果

目的

ScreenTrace 是什麼

輸入

專案原始碼

JSP 、 Java 、 XML 、
JS ／ TS 、 React

ScreenTrace

./bin/screentrace
。專案路徑。

靜態分析，不啟動系統

離線 HTML 報表

畫面地圖、預覽、試操作、 API 頁

application-graph.json

畫面、按鈕、 API 的關係與來源

決策 md

保留／移除的標記與分析限制

讀 **唯讀**

不啟動、不執行、不連資料庫、不改原始碼

溯 **可追溯**

每條關係都記錄 file:line 與來源 parser

證 **不猜測**

證明不了就標成推導、歧義或未解析

功能

它能幫你回答什麼

01

畫面地圖

有哪些畫面？彼此怎麼連？

報表位置：依網址分組的畫布

02

畫面預覽

不啟動系統，也能看到畫面長相

報表位置：靜態重建＋截圖

03

按鈕去向

這顆按鈕按下去會到哪個畫面？

報表位置：右側按鈕清單

04

API 呼叫

這個畫面、這顆按鈕打了哪些 API ？

報表位置：呼叫的 API

05

API 反查與契約

這支 API 被誰呼叫？參數是什麼？

報表位置： API 頁

06

保留／移除決策

標記後哪些 API 變成可移除？

報表位置：標記＋下載 md

怎麼使用

使用步驟 ①：一個指令完成分析

```
# 第一次：安裝與檢查環境
./bin/screentrace setup

# 分析專案並開啟報表
./bin/screentrace ~/projects/reservio

# Windows
.\bin\screentrace.cmd C:\projects\reservio
```

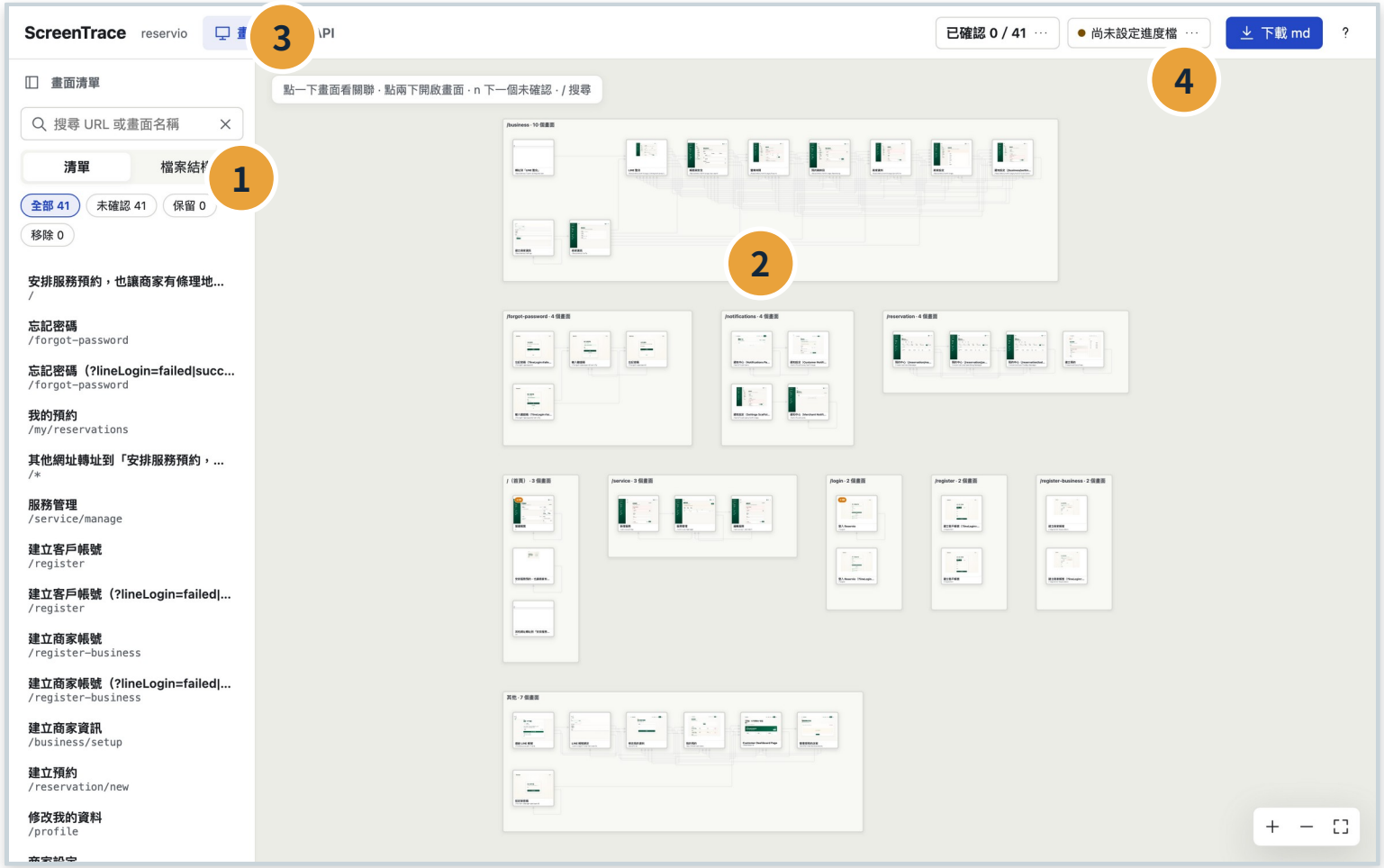
產出位置

```
analyze/reservio/
├─ report/
│  └─ screentrace-report.html
├─ application-graph.json
├─ screenshots/
└─ static-preview/
```

報表是單一 HTML 檔：可以直接寄給需求方，雙擊用瀏覽器開啟，不需要伺服器或網路

怎麼使用

使用步驟 ②：先看整個系統有哪些畫面



1 **畫面清單與搜尋**

輸入網址或名稱，快速找到畫面

2 **畫面地圖**

依網址分組；線代表畫面之間的去向

3 **畫面／API 頁籤**

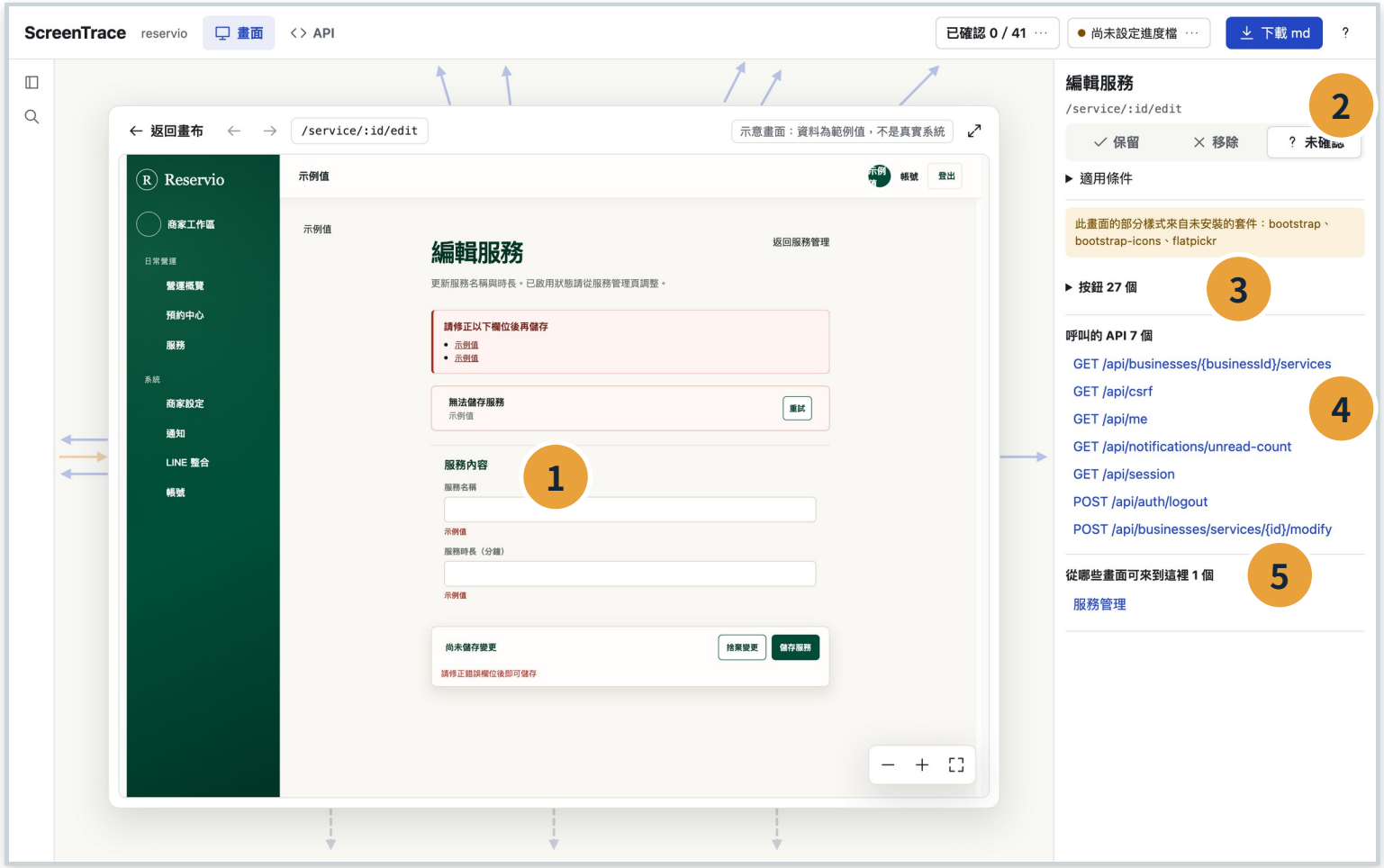
從畫面看，或從 API 反查

4 **進度與下載 md**

已確認幾個畫面、匯出決策

怎麼使用

使用步驟 ③：打開畫面，看按鈕與 API



1 **預覽與試操作**

點按鈕會依分析結果換頁

2 **保留／移除標記**

需求方對這個畫面的決定

3 **按鈕清單**

每顆按鈕的 API 與去向

4 **呼叫的 API**

這個畫面用到的所有 API

5 **從哪些畫面來**

反向的進入路徑

怎麼使用

使用步驟 ④：從 API 反查，最後交付 md

使用中

至少一個呼叫來源仍保留或未確認

可移除

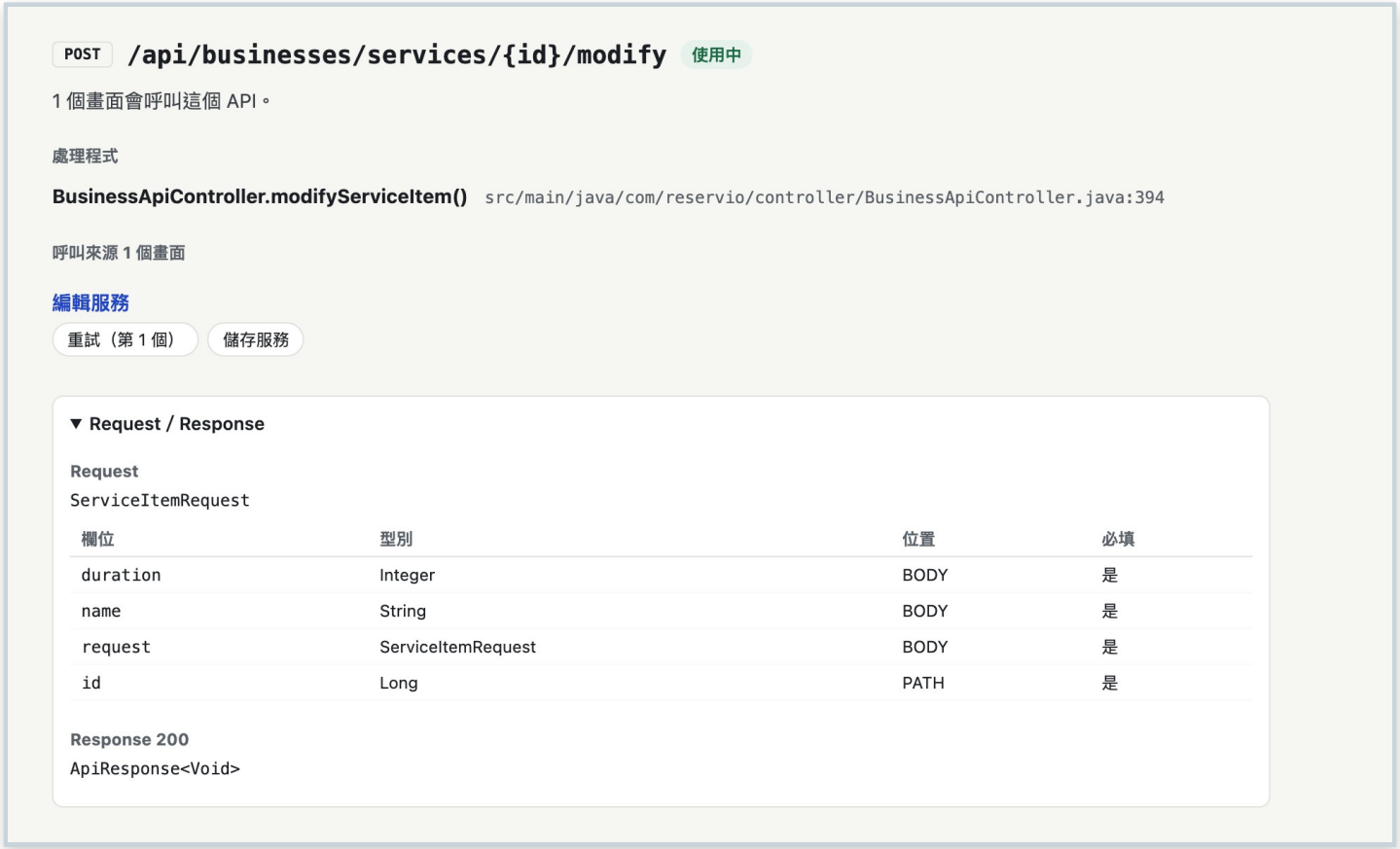
所有呼叫來源都已標記移除

未被使用

畫面中找不到呼叫來源（外部系統仍可能使用）

下載 md

決策、來源與分析限制，交給工程或 AI 接手



reservio API 頁：處理程式、呼叫來源與 Request ／ Response

功能

支援範圍與邊界

支援

- Struts 1
- Struts ＋ Spring
- Spring MVC （ JSP ）
- Spring Boot （ JSP ）
- Spring Boot ＋ React （ react-router ）

部分支援

- JSF 頁面：標記為未分析
- Thymeleaf ：有限的 URL 與片段
- 動態 URL 、反射：保留為未解析

不在範圍

- Struts 2 ：直接拒絕分析
- Angular 、 Vue ：後續方向
- 自動轉換或修改程式碼

原則：分析不了的部分明確標出來，不產生「看起來完整」的錯誤結果

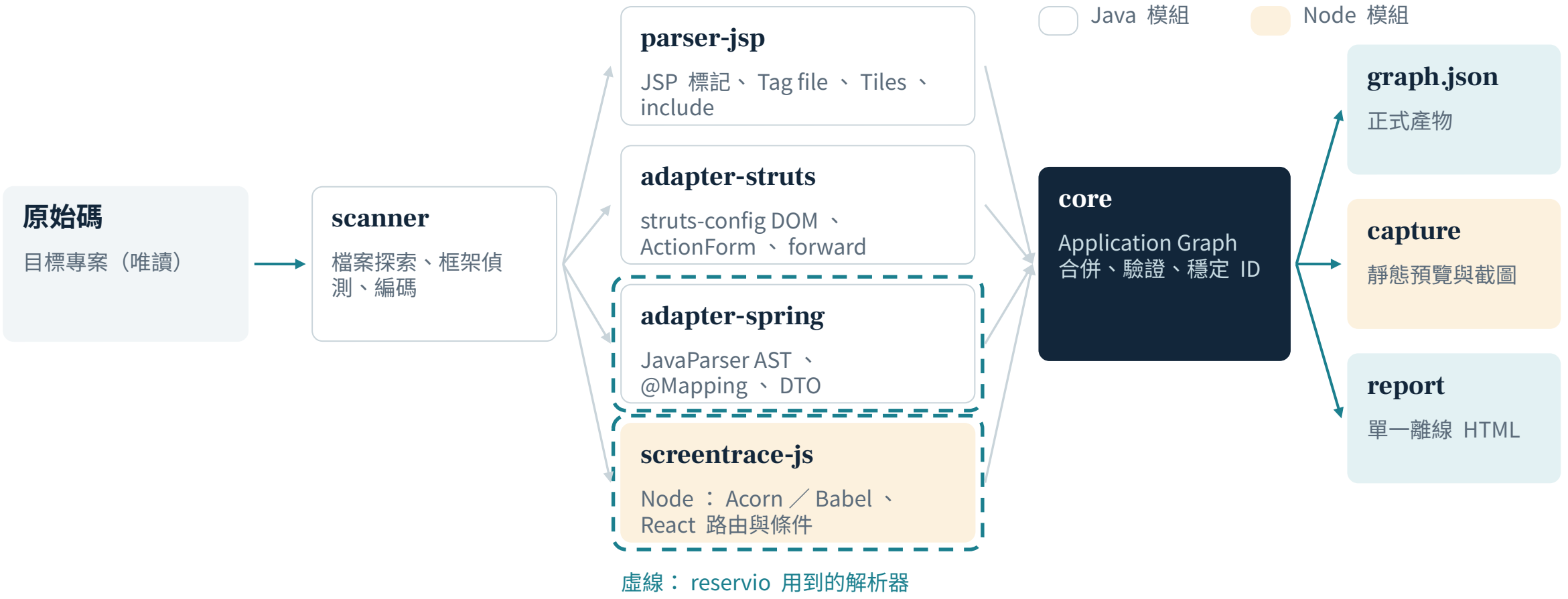
02

技術架構

原始碼怎麼變成一張可追溯的圖

技術架構

模組架構與資料流



screentrace-cli　協調整個流程：分析 → 預覽 → 封裝報表 → 開啟瀏覽器

技術架構

分析管線：五個階段

01

探索

列出檔案
偵測框架與技術訊號

scanner

02

解析

XML 、 Java 、
JSP 、
JS ／ TS 各自解析

parser ／ adapter

03

關聯

URL 、 method 、
view 、
forward 互相配對

adapter

04

驗證

合併各解析結果
檢查圖完整性與來源

core

05

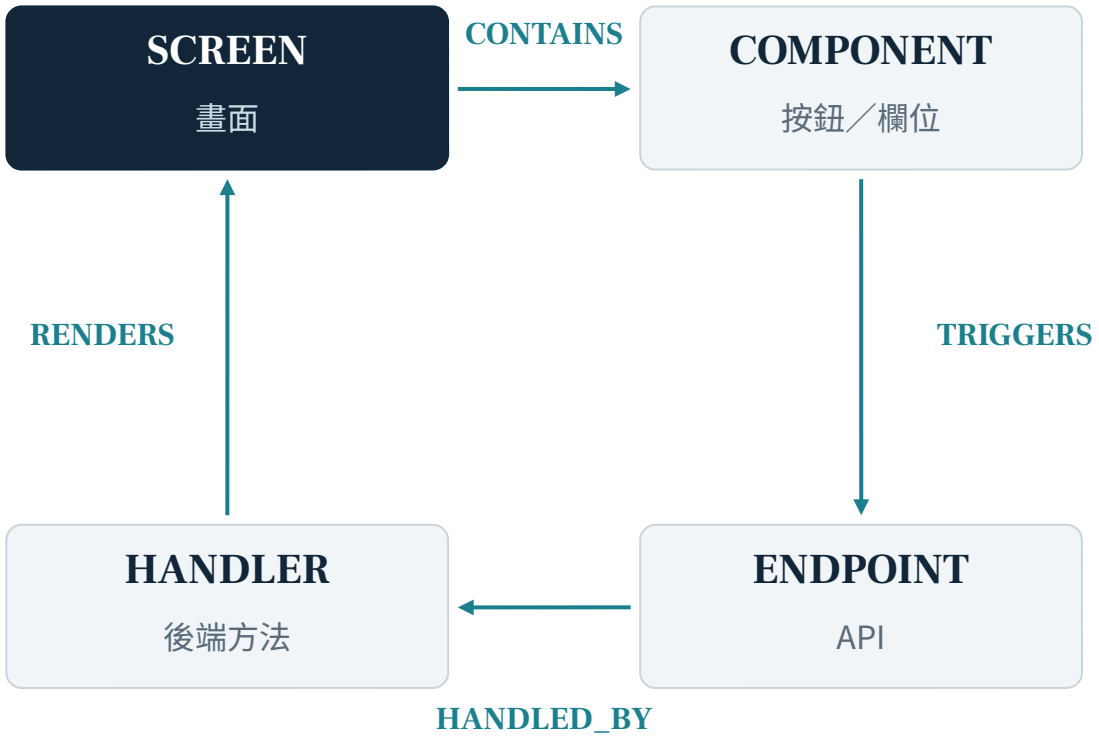
重建與封裝

靜態預覽、截圖
打包成單一 HTML

capture ／ report

關鍵在第 3 步：前端送出的 URL 要對上後端的 mapping ，才成為一條完整流程

Application Graph ：所有輸出共用的一張圖



COMPONENT —NAVIGATES_TO→ SCREEN ：按鈕前往的下一個畫面


```
{
  "type": "HANDLED_BY",
  "from": "POST /api/businesses/
          services/{id}/modify",
  "to":   "BusinessApiController
          .modifyServiceItem()",
  "confidence": "CONFIRMED",
  "evidence": [{
    "source": { "file": ".../
      BusinessApiController.java",
      "line": 394 },
    "parser": "SpringBootAnalyzer" }]
}
```

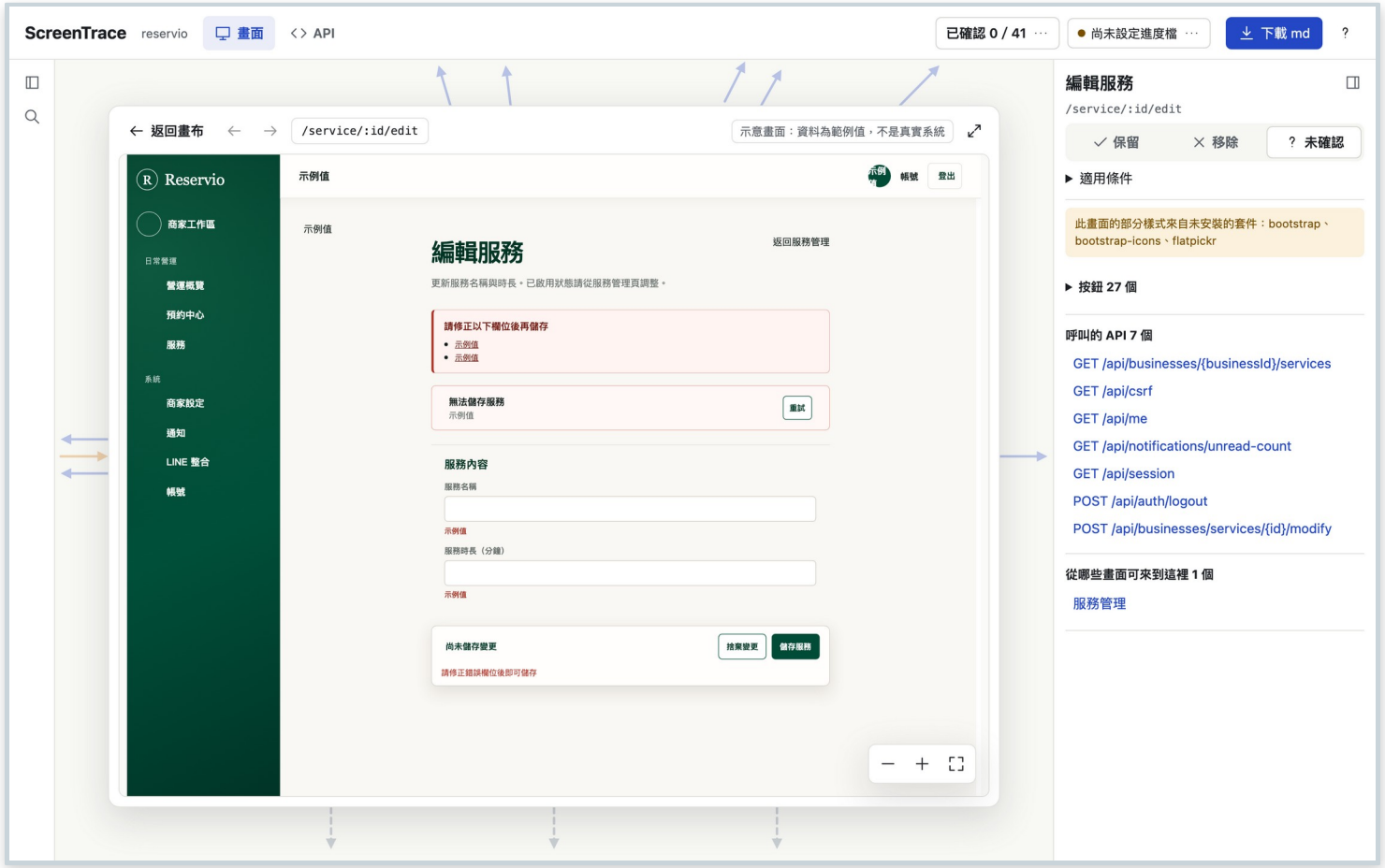

reservio 的實際關係（ID 已換成名稱）

03

實例：拆解 reservio 「編輯服務」

ScreenTrace 怎麼從原始碼找出這個畫面的來龍去脈

今天的例子： /service/:id/edit



1. **這個畫面怎麼來的？**
   路由、身分條件、畫面元件

2. **畫面上有什麼？**
   欄位、按鈕、連結

3. **按鈕打了哪支 API ？**
   前端呼叫 → 後端方法 → 參數

4. **從哪來、能去哪？**
   進入路徑與離開路徑

實例：編輯服務

① 這個畫面怎麼來的

```
// frontend/src/App.tsx
function RequireBusiness({ children }) {        // L107
  const { me, loading } = useAuth();
  if (!me) return <Navigate to={loginPath}/>;
  if (me.mustChangePassword
      || me.role !== 'ROLE_BUSINESS'
      || !me.business)                          // L112
    return <Navigate .../>;
  return children;
}

<Route path="/service/:id/edit"                 // L273
  element={<RequireAuth><RequireBusiness>
    <ServiceEditorScaffold editing/> ...}/>
```

ScreenTrace 產生

ENTRY_POINT — App.tsx:273

/service/:id/edit

CONFIRMED

SCREEN — RENDERS ← 入口

編輯服務

INFERRED

適用條件 — App.tsx:112

ROLE_BUSINESS 、有 business 、不需強制改密碼

INFERRED

畫面元件 — SettingsPages.tsx:48

ServiceEditorScaffold
→ ServiceEditorPage.tsx

② 畫面上有什麼

```
// ServiceEditorPage.tsx  L22 （節錄）
<SettingsPageHeader title=" 編輯服務 "
  actions={<Link to="/service/manage">
           返回服務管理 </Link>}/>
<input id="service-name" .../>
<input id="service-duration" type="number"/>
<StickySaveBar onSave={() => void save()}
  onDiscard={form.discard}
  primaryLabel={editing ? ' 儲存服務 ' : ...}/>
```

30 個 COMPONENT （圖）／ 27 個按鈕與連結（報表）

本頁內容

- 服務名稱、服務時長（ 2 個欄位）
- 儲存服務、捨棄變更
- 返回服務管理（連結）

共用外框

- 側邊欄：營運概覽、預約中心、服務、商家設定…
- 頂部列：通知、帳號、登出

狀態元件

- 載入失敗時的「重試」
- 登入逾時提醒的對話框

③ 按下「儲存服務」，打了哪支 API

1	按鈕	「儲存服務」	ServiceEditorPage.tsx:22
2	事件處理	save()	ServiceEditorPage.tsx:19
3	API 函式	api.modifyServiceItem(id, payload)	ServiceEditorPage.tsx:19
4	組出請求	request(`/api/businesses/services/${id}/modify`, { method: 'POST' })	api.ts:241

TRIGGERS → POST /api/businesses/services/{id}/modify

INFERRED

同一顆按鈕還記錄了

GET /api/csrf
送 POST 前取 token

GET …/services
存完重新讀取

→ 服務管理
成功後換頁

③ 前端的 URL ，怎麼對上後端的方法

前端（ screentrace-js ）

api.ts:241

POST
/api/businesses/services/
${id}/modify

正規化 → /api/businesses/services/*/modify

=

method
＋路徑樣板

```
// BusinessApiController.java
@RequestMapping("/api")                    // L23
public class BusinessApiController {
  @PostMapping("/businesses/services/{id}/modify")
  public ResponseEntity<ApiResponse<Void>>  // L394
      modifyServiceItem(
        @PathVariable Long id,
        @RequestBody ServiceItemRequest request)
```

HANDLED_BY

CONFIRMED

BusinessApiController
.modifyServiceItem()

Request

id: Long （ PATH ）
name: String 、 duration: Integer

Response 200

ApiResponse<Void>

④ 從哪裡來，可以去到哪裡

從哪裡來

服務管理「編輯」

<Link to={`/service/${id}/edit`}>

ServiceManagementPage.tsx:18

INFERRED

編輯服務

/service/:id/edit

進入時 GET …/{businessId}/services

可以去到哪裡

儲存服務 → 服務管理

navigate('/service/manage') · L19

返回服務管理 → 服務管理

<Link to="/service/manage"> · L22

側邊欄 → 營運概覽、預約中心…

DesktopWorkspaceSidebar.tsx:31

登出 → 登入

POST /api/auth/logout · LogoutButton.tsx

樣板組出的網址標 INFERRED ，字面值標 CONFIRMED ：每條線都說明自己有多確定

合起來：報表上看到的，就是這一整條線

服務管理 畫面 — ServiceManagementPage.tsx:18

↓ NAVIGATES_TO

編輯服務 畫面 /service/:id/edit — App.tsx:273

↓ CONTAINS

儲存服務 按鈕 — ServiceEditorPage.tsx:22

↓ TRIGGERS

POST …/services/{id}/modify API — api.ts:241

↓ HANDLED_BY

modifyServiceItem() 後端方法 — BusinessApiController.java:394

↓ NAVIGATES_TO

服務管理 下一個畫面 — ServiceEditorPage.tsx:19



報表右側的按鈕清單

實例：編輯服務

每條關係都標明「有多確定」

CONFIRMED

原始碼直接證明

@PostMapping(.../modify)
→ modifyServiceItem()

2,520

reservio ・ 條關係

INFERRED

依明確規則推導

`/api/.../${id}/modify`
→ 修改 API

294

reservio ・ 條關係

AMBIGUOUS

多個候選，全部保留

「返回首頁」→ /
訪客首頁或商家首頁

34

reservio ・ 條關係

UNRESOLVED

無法證明，保留原文

LINE 登入導回路徑
SecurityConfig.java:104

5

reservio ・ 筆診斷

目前的限制，以及需要後端一起補的地方

靜態分析看不到的

- 反射、動態 SQL 、執行期組出的 URL 無法保證還原
- 預覽用示例資料；缺套件 CSS 或未解析邏輯會讓畫面有差異
- 外部系統、排程、書籤、 Webhook 的呼叫看不到
- 只做盤點與決策交付，不自動轉換或修改程式

需要後端同仁協助的

- 核對 Controller 的回傳分支與導向是否完整
- 補上外部呼叫來源：批次、其他系統、第三方回呼
- 提供 context path 等部署設定，減少未解析 URL
- 挑一條真實功能流程，一起驗證結果

靜態預覽是「示意」，不等於原系統行為；決策前仍需工程確認

下一步

挑一個內部專案，驗證一條完整功能流程

1. 畫面與按鈕的去向是否正確？
2. Controller 的分支與 API 契約是否完整？
3. 有哪些外部入口需要補上？

討論 · Q&A